Timoni + CUE, alongside Flux
Author → render → reconcile

Tool	Responsibility
CUE + Timoni	Define constrained inputs; build and package objects.
Kustomize	Transform YAML using bases, patches and replacements.
Flux	Continuously reconcile desired state in Kubernetes.

What changes when configuration has an explicit **contract**?

Sources: Timoni · Kustomize · Flux

CUE's creator: Marcel van Lohuizen

- Member of the founding **Borg** team at Google.
- Created **BCL**, the Borg Configuration Language, later generalized into GCL.
- Created **CUE**, drawing on experience with large configurations.

Borg / BCL → GCL → CUE

The design idea: combine **data and constraints** in one language.

Sources: Marcel's biography · CUE history

Types and values share one model

Think of a constraint as a **set of allowed values**.

```
replicas: *2 | (int & ⩾1 & ⩽10)
```

- `int & ⩾1 & ⩽10`: integers in the intersection of these rules.
- `|` offers alternatives; `*2` marks the preferred default.

No input → **2**

Input `3` → **3**

Input `0` → **error**

Sources: Constraints · Defaults

Composition means satisfying both

Reusable definition + concrete input

```
#Config: {
  replicas: int & ⩾1 & ⩽10
}

a: #Config & {replicas: 3}
b: {replicas: 3} & #Config
```

Unification

Both `a` and `b` resolve to:

```
replicas: 3
```

`A & B = B & A`

Conflicting information produces an error.

`#Config` names reusable structure and constraints.

Sources: Unification · Definitions · Composition properties

Transform YAML or constrain inputs

Kustomize · kustomization.yaml excerpt

```yaml
resources:
- ../base
replicas:
- name: demo
  count: 3
```

CUE · definition + input

```
#Config: {
  replicas: *2 | (int & ⩾1 & ⩽10)
}
values: #Config & {replicas: 3}
```

Input	Kustomize result	CUE module result
`3`	Render `3`	Accept `3`
`0`	Render `0`	Reject: outside `1–10`

Kustomize can be paired with separate schema and policy checks.

Sources: Kustomize operations · Offline comparison

Timoni packages CUE modules

- **Interface:** values, defaults and constraints in a module.
- **Schemas:** vendor Kubernetes API and CRD definitions for validation.
- **Distribution:** versioned CUE source modules in OCI registries.

Build and validate the local WebApp module

```
timoni build rgd-demo ./example-rgd -n demo
timoni mod vet ./example-rgd --name rgd-demo -n demo
```

Build produces YAML. Timoni also offers CLI deployment and inventory.

Sources: Timoni concepts · Modules · CRD schemas

Use shared CUE structure for a kro API

CUE · schema + shared conventions

```
#RGD:
  rgdv1.#ResourceGraphDefinition & {
    spec: schema: {
      group: "platform.example"
      scope: "Namespaced"
    }
  }
#WebAppRGD: #RGD & {
  metadata: name: "web-app"
  // API + resource templates
}
```

Rendered RGD · schema/template excerpt

```
schema:
  kind: WebApp
  spec:
    replicas: >-
      integer | default=2
      minimum=1 maximum=10
resources:
- id: webDeployment
  template:
    kind: Deployment
    spec:
      replicas: >-
        ${schema.spec.replicas}
```

Timoni sets the API default **2**; kro uses each instance's value.

Sources: kro v0.9.4 · Complete module, including vendored schema · Flux values handoff

Where Bicep values meet Kubernetes

Existing infrastructure handoff

Bicep / CI → ConfigMaps / Secrets → Flux → Kubernetes

Timoni Runtime → typed Bundle inputs · excerpt (separate CLI step)

```
bundle: {
  _replicas: int    @timoni(runtime:number:DEMO_REPLICAS)
  _token:    string @timoni(runtime:string:DEMO_TOKEN)
  instances: web: values: {
    replicas: int & ⩾1 & ⩽10 & _replicas
    token:    string & ≠"" & _token
  }
}
```

`"3"` → integer `3`; `0`, `"three"` or an empty token → **error**.

Sources: Flux substitution · Timoni Runtime · Offline handoff example

Choose the abstraction you need

What CUE + Timoni offer

- Explicit, reusable configuration contracts.
- Constraint composition and errors before deployment.
- Modules with schema checks and OCI distribution.

What the team takes on

- CUE's model and error diagnostics.
- Module, schema and toolchain maintenance.
- Validation at the external-values handoff.

Use Kustomize for straightforward transformations. Consider CUE when shared rules need to compose.

Try it: Deployment · Kustomize comparison · WebApp RGD